

2024最新react面试题（中高级开发31题）

1.React19新特性？

- 代码小迷妹
- 代码小迷妹
- 代码小迷妹
1. **改进的 Concurrent Rendering（并发渲染）**： 并发渲染技术进一步优化，允许 React 应用在后台更新状态时，不会阻塞用户界面的交互，从而提高响应速度和用户体验。
 2. **增强的 Automatic Batching（自动批处理）**： React 19 可能进一步提升自动批处理的功能，减少不必要的重新渲染次数，优化应用的性能。
 3. **强化的 Hooks 支持**： React 19 或将引入更多自定义 Hooks，如 `useTransition` 和 `useDeferredValue`，帮助开发者更灵活地控制 UI 更新的时机，提升应用的流畅度。
 4. **更强大的 Suspense for Data Fetching（数据获取的 Suspense 支持）**： 数据获取与异步操作的 Suspense 机制得到进一步增强，使得处理异步数据更加自然和高效。
 5. **改进的生命周期方法替代方案**： `useEffect` 钩子在提供与类组件生命周期方法相近的功能时，可能会进一步优化，提供更高效率的组件挂载和更新逻辑。v

2.React组件通信的方式有哪些？

- 代码小迷妹
- 代码小迷妹
- 代码小迷妹
- **父组件向子组件通讯**: 父组件可以向子组件通过传 props 的方式，向子组件进行通讯
 - **子组件向父组件通讯**: props+回调的方式，父组件向子组件传递props进行通讯，此props为作用域为父组件自身的函数，子组件调用该函数，将子组件想要传递的信息，作为参数，传递到父组件的作用域中。
 - **兄弟组件通信**: 找到这两个兄弟节点共同的父节点,进行状态提升，结合上面两种方式由父节点转发信息进行通信
 - **跨层级通信**:
 - props层层传递回调函数。
 - Context 设计目的是为了共享那些对于一个组件树而言是“全局”的数据，例如当前认证的用户、主题或首选语言，对于跨越多层的全局数据通过 Context 通信再适合不过。
 - **非嵌套关系的组件通信**:
 - (发布订阅模式): 发布者发布事件，订阅者监听事件并做出反应,我们可以通过引入event模块进行通信
 - (全局状态管理工具): 借助Redux或者Mobx等全局状态管理工具进行通信,这种工具会维护一个全局状态中心Store,并根据不同的事件产生新的状态

3.简述React的生命周期

React 的生命周期主要分为三个阶段：MOUNTING、RECEIVE_PROPS、UNMOUNTING

- 组件挂载时（组件状态的初始化，读取初始 state 和 props 以及两个生命周期方法，只会在初始化时运行一次）
 - componentWillMount 会在 render 之前调用（在此调用 setState，是不会触发 re-render 的，而是会进行 state 的合并。因此此时的 this.state 不是最新的，在 render 中才可以获取更新后的 this.state。）
 - componentDidMount 会在 render 之后调用
- 组件更新时（组件的更新过程是指父组件向下传递 props 或者组件自身执行 setState 方法时发生的一系列更新的动作）
 - 组件自身的 state 更新，依次执行
 - shouldComponentUpdate（会接收需要更新的 props 和 state，让开发者增加必要的判断条件，在其需要的时候更新，不需要的时候不更新。如果返回的是 false，那么组件就不再向下执行生命周期方法。）
 - componentWillMount
 - render（能获取到最新的 this.state)
 - componentDidUpdate（能获取到最新的 this.state)
 - 父组件更新 props 而更新
 - componentWillReceiveProps（在此调用 setState，是不会触发 re-render 的，而是会进行 state 的合并。因此此时的 this.state 不是最新的，在 render 中才可以获取更新后的 this.state。
 - shouldComponentUpdate
 - componentWillMount
 - render
 - componentDidUpdate
- 组件卸载时
 - componentWillUnmount（我们常常会在组件的卸载过程中执行一些清理方法，比如事件回收、清空定时器）

新版的生命周期函数增加了 getDerivedStateFromProps，这个生命周期其实就是将传入的 props 映射到 state 中。在 React 16.4 之后，这个函数每次会在 re-render 之前调用，

getDerivedStateFromProps的作用是

- 无条件的根据 prop 来更新内部 state，也就是只要有传入 prop 值， 就更新 state
- 只有 prop 值和 state 值不同时才更新 state 值。

但是盲目使用这个生命周期会有一些问题

- 直接复制 props 到 state 上
- 如果 props 和 state 不一致就更新 state

4.React事件机制和原生DOM事件流有什么区别

react中的事件是绑定到document上面的，

而原生的事件是绑定到dom上面的，

代码小迷妹

因此相对绑定的地方来说，dom上的事件要优先于document上的事件执行

5.Redux工作原理

Redux 是 React 的第三方状态管理库，创建于上下文API存在之前。它基于一个称为存储的状态容器的概念，组件可以从该容器中作为 props 接收数据。更新存储区的唯一方法是向存储区发送一个操作，该操作被传递到一个reducer中。reducer接收操作和当前状态，并返回一个新状态，触发订阅的组件重新渲染。

代码小迷妹

代码小迷妹

代码小迷妹

6.React-Router工作原理? react-router-dom有哪些组件

路由器组件，路由匹配组件，导航组件

react-router 的依赖库history

- 代码小迷妹
- 1 1、BrowserHistory：用于支持 HTML5 历史记录 API 的现代 Web 浏览器（请参阅跨浏览器兼容性） \
- 2 2、HashHistory：用于旧版Web浏览器\
- 3 3、MemoryHistory：用作参考实现，也可用于非 DOM 环境，如 React Native 或测试
- 4
- 5 BrowserHistory: pushState、replaceState
- 6 HashHistory: location.hash、location.replace

代码小迷妹

代码小迷妹

代码小迷妹

7.React hooks解决了什么问题? 函数组件与类组件的区别

Hooks 解决了我们五年来编写和维护成千上万的组件时遇到的各种各样看起来不相关的问题。无论你正在学习 React，或每天使用，或者更愿尝试另一个和 React 有相似组件模型的框架，你都可能对这些问题似曾相识。

在 React 16.8版本（引入钩子）之前，使用基于类的组件来创建需要维护内部状态或利用生命周期方法的组件（即 `componentDidMount` 和 `shouldComponentUpdate` ）。基于类的组件是 ES6 类，它扩展了 React 的 `Component` 类，并且至少实现了 `render()` 方法。

代码小迷妹

代码小迷妹

代码小迷妹

类组件：

```
1 class Welcome extends React.Component {
2   render() {
```


代码小迷妹

代码小迷妹

代码小迷妹

```
3     return <h1>Hello, {this.props.name}</h1>;
4   }
5 }
```

函数组件是无状态的（同样，小于 React 16.8版本），并返回要呈现的输出。它们渲染 UI 的首选只依赖于属性，因为它们比基于类的组件更简单、更具性能。

代码小迷妹

代码小迷妹

代码小迷妹

函数组件：

```
1 function Welcome(props) {
2   return <h1>Hello, {props.name}</h1>;
3 }
```

代码小迷妹

代码小迷妹

代码小迷妹

注意：在 React 16.8版本中引入钩子意味着这些区别不再适用（请参阅14和15题）。

8.SetState是同步还是异步的，setState做了什么

在React中，`setState()` 函数通常被认为是异步的，这意味着调用`setState()`时不会立刻改变react 组件中state的值，`setState`通过触发一次组件的更新来引发重汇，多次`setState`函数调用产生的效果会合并

调用 `setState`时，React会做的第一件事情是将传递给 `setState`的对象合并到组件的当前状态。这将启动一个称为和解（reconciliation）的过程。和解（reconciliation）的最终目标是以最有效的方式，根据这个新的状态来更新UI。为此，React将构建一个新的 React元素树（您可以将其视为 UI 的对象表示）。

一旦有了这个树，为了弄清 UI 如何响应新的状态而改变，React 会将这个新树与上一个元素树相比较

9.什么是fiber，fiber解决了什么问题

代码小迷妹

代码小迷妹

代码小迷妹

React15 的 `StackReconciler` 方案由于递归不可中断问题，如果 Diff 时间过长（JS计算时间），会造成页面 UI 的无响应（比如输入框）的表现，`vdom` 无法应用到 `dom` 中。

为了解决这个问题，React16 实现了新的基于 `requestIdleCallback` 的调度器（因为 `requestIdleCallback` 兼容性和稳定性问题，自己实现了 `polyfill`），通过任务优先级的思想，在高优先级任务进入的时候，中断 `reconciler`。

为了适配这种新的调度器，推出了 `FiberReconciler`，将原来的树形结构（vdom）转换成 Fiber 链表的形式（child/sibling/return），整个 Fiber 的遍历是基于循环而非递归，可以随时中断。

代码小迷妹

代码小迷妹

代码小迷妹

更加核心的是，基于 Fiber 的链表结构，对于后续（React 17 lane 架构）的异步渲染和（可能存在的）worker 计算都有非常好的应用基础

10.React中在哪捕获错误？

官网例子：


```
1 class ErrorBoundary extends React.Component {
2   constructor(props) {
3     super(props);
4     this.state = { hasError: false };
5   }
6
7   static getDerivedStateFromError(error) {
8     // 更新 state 使下一次渲染能够显示降级后的 UI
9     return { hasError: true };
10  }
11
12  componentDidCatch(error, errorInfo) {
13    // 你同样可以将错误日志上报给服务器
14    logErrorToMyService(error, errorInfo);
15  }
16
17  render() {
18    if (this.state.hasError) {
19      // 你可以自定义降级后的 UI 并渲染
20      return <h1>Something went wrong.</h1>;
21    }
22
23    return this.props.children;
24  }
25 }
```

使用

```
1 <ErrorBoundary>
2   <MyWidget />
3 </ErrorBoundary>
```

但是错误边界不会捕获：

```
1 try {} catch(err) {}
2 /// 异步代码（例如 setTimeout 或 requestAnimationFrame 回调函数）
3 /// 服务端渲染
4 /// 它自身抛出来的错误（并非它的子组件）
```

1. React组件传值有哪些方式

父传子： props 子传父： 通过在父组件引入的子组件中传递一个函数并传参数，子组件去触发这个函数更改参数完成数据更新

跨多层组件传值： 通过context api完成

2. react无状态组件和class类组件的区别

- 直观区别，函数组件代码量较少，相比类组件更加简洁
- 函数组件看似只是一个返回react元素的函数，其实体现的是无状态组件的思想，函数组件中没有this， 没有state，也没有生命周期，这就决定了函数组件都是展示性组件，接收props，渲染dom，而不关注其他逻辑
- 因为函数组件不需要考虑组件状态和组件生命周期方法中的各种比较校验，所以有很大的性能提升空间

13.react如何做到和vue中keep-alive的缓存效果

React Keep Alive 提供了 ，你必须把 放在 Provider 里面，并且每个 组件都必须拥有一个唯一的 key

14.React如何做路由监听

componentDidMount(){ this.context.router.history.listen((route)=>{ if(route.pathname==='/xxx'){ console.log(1); } }); } 还可以用高阶组件

15.React有哪几种方式改变state

this.forceUpdate this.setState :key值传递不同也可以 replaceState也可以改变

16.React有哪几种创建组件方法

React 有三种构建组件的方式

- React.createClass
- ES6 class
- 无状态函数

React.createClass 是 React 最传统、兼容性最好的方法。该方法构建一个组件对象, 当组件被调用时，就会创建几个组件实例

ES6 class 方式和 createClass 类似，只是从调用内部方法变成了用类来实现。

无状态组件创建时始终保持一个实例，避免了不必要的检查和内存分配。

17.react中props和state有什么区别

props 是传递给组件的（类似于函数的形参），而 state 是在组件内被组件自己管理的（类似于在一个函数内声明的变量）

props 是不可修改的，所有 React 组件都必须像纯函数一样保护它们的 props 不被更改。 由于 props 是传入的，并且它们不能更改，因此我们可以将任何仅使用 props 的 React 组件视为 pureComponent，也就是说，在相同的输入下，它将始终呈现相同的输出。 state 是在组件中创建的，一般在 constructor中初始化 state state 是多变的、可以修改，每次setState都异步更新的。

18.React 中 keys 的作用是什么？

Keys 是 React 用于追踪哪些列表中元素被修改、被添加或者被移除的辅助标识。

在 React 中渲染集合时，向每个重复的元素添加关键字对于帮助React跟踪元素与数据之间的关联非常重要。key 应该是唯一ID，最好是 UUID 或收集项中的其他唯一字符串：

```
1 <ul>
2   {todos.map((todo) =>
3     <li key={todo.id}>
4       {todo.text}
5     </li>
6   )};
7 </ul>
```

在集合中添加和删除项目时，不使用键或将索引用作键会导致奇怪的行为。

19.React 中 refs 的作用是什么？

Refs 是 React 提供给我们的安全访问 DOM 元素或者某个组件实例的句柄。

我们可以为元素添加 ref 属性然后在回调函数中接受该元素在 DOM 树中的句柄，该值会作为回调函数的第一个参数返回：

```
1 class CustomForm extends Component {
2   handleSubmit = () => {
3     console.log("Input Value: ", this.input.value)
4   }
5   render () {
6     return (
7       <form onSubmit={this.handleSubmit}>
8         <input
9           type='text'
10          ref={(input) => this.input = input} />
11         <button type='submit'>Submit</button>
12       </form>
13     )
14   }
```



```
15 }
```

上述代码中的 input 域包含了一个 ref 属性，该属性声明的回调函数会接收 input 对应的 DOM 元素，我们将其绑定到 this 指针以便在其他的类函数中使用。

另外值得一提的是，refs 并不是类组件的专属，函数式组件同样能够利用闭包暂存其值：

```
1 function CustomForm ({handleSubmit}) {
2   let inputElement
3   return (
4     <form onSubmit={() => handleSubmit(inputElement.value)}>
5       <input
6         type='text'
7         ref={(input) => inputElement = input} />
8     <button type='submit'>Submit</button>
9   </form>
10 )
11 }
```

20.React diff 原理

- 把树形结构按照层级分解，只比较同级元素。
- 列表结构的每个单元添加唯一的 key 属性，方便比较。
- React 只会匹配相同 class 的 component（这里面的 class 指的是组件的名字）
- 合并操作，调用 component 的 setState 方法的时候, React 将其标记为 dirty 到每一个事件循环结束, React 检查所有标记 dirty 的 component 重新绘制。
- 选择性子树渲染。开发人员可以重写 shouldComponentUpdate 提高 diff 的性能。

21.受控组件和非受控组件有什么区别？

在 HTML 文档中，许多表单元素（例如 、☐）都保持自己的状态。不受控制的组件将 DOM 视为这些输入状态的真实源。在受控组件中，内部状态用于跟踪元素值。当输入值改变时，React 会重新渲染输入。

在与非 React 代码集成时，不受控制的组件非常有用（例如，如果您需要支持某种 jQuery 表单插件）。

3. 为什么虚拟 dom 会提高性能？

虚拟dom相当于在js和真实dom中间加了一个缓存，利用dom diff算法避免了没有必要的dom操作，从而提高性能。

具体实现步骤如下：

用 JavaScript 对象结构表示 DOM 树的结构；然后用这个树构建一个真正的 DOM 树，插到文档当中。当状态变更的时候，重新构造一棵新的对象树。然后用新的树和旧的树进行比较，记录两棵树差异。把2所记录的差异应用到步骤1所构建的真正的DOM树上，视图就更新了。

22.Redux 和 Vuex 有什么区别，它们的共同思想？

代码小迷妹

Redux Redux 是一个用于 JavaScript 应用的状态管理库，广泛用于 React（也可以用于其他框架或库）：

代码小迷妹

代码小迷妹

- 单一状态树（Store）： Redux 使用一个单一的、不可变的状态树来存储整个应用的状态。
- Action： 状态更改是通过发送（dispatch） action 来表示的。Action 是描述“发生了什么”的普通 JavaScript 对象。
- Reducer： Reducer 函数决定如何根据 action 更改状态。Reducer 必须是纯函数，它接收前一个状态和一个 action，返回新状态。
- 中间件： Redux 支持中间件，用于处理副作用（如异步操作或日志记录）。

代码小迷妹

代码小迷妹

代码小迷妹

Vuex Vuex 是专为 Vue.js 设计的状态管理模式和库：

- 状态管理（State）： Vuex 也使用单一状态树，存储整个应用的状态。
- Mutation： 更改 Vuex 的状态的唯一方法是通过提交（commit） mutation。Mutation 必须是同步函数。
- Action： Action 类似于 Redux 中的 action，用于提交 mutation。不同的是，它们可以包含异步操作。
- Module： Vuex 允许将 store 分割成模块，每个模块可以拥有自己的状态、mutation、action 和 getter。

代码小迷妹

代码小迷妹

代码小迷妹

共同思想 尽管 Redux 和 Vuex 的实现细节有所不同，但它们在状态管理方面有着共同的理念：

- **集中式状态管理**： 它们都提倡将应用的状态集中存储在单一的状态树中，这使得状态更容易追踪和调试。
- **可预测性**： 通过明确的规则和模式（如 Redux 的 reducer 或 Vuex 的 mutation）来管理状态更改，使得应用的状态变得可预测和一致。
- **维护性和组织性**： 通过提供一种清晰的方式组织代码和管理数据流，帮助开发者构建大型、复杂的应用程序。

代码小迷妹

代码小迷妹

代码小迷妹

主要区别

- **框架依赖**： Redux 是一个独立的库，可以与任何 UI 层一起使用，但通常与 React 一起使用；Vuex 专为 Vue.js 设计。
- **异步操作**： Redux 本身不支持异步操作，需要中间件（如 redux-thunk 或 redux-saga）；Vuex 的 action 支持异步操作。
- **API 和设计哲学**： Redux 倾向于函数式编程风格，Vuex 更紧密地集成了 Vue.js 的响应式系统。

代码小迷妹

代码小迷妹

代码小迷妹

23. 循环渲染中 为什么推荐不用 index 做 key?

关键词：React 循环渲染问题

在 React 的循环渲染中，不推荐使用数组的 `index`（索引）作为元素的 `key`，主要基于以下几点理由：

1. **列表项顺序的变化：**如果列表项的顺序会发生变化（如排序、增加、删除操作），使用 `index` 作为 `key` 可能会导致性能问题和组件状态的错误。这是因为 React 依赖 `key` 来判断哪些元素是新元素、哪些被移除，以及哪些元素的位置发生了变化。当使用 `index` 作为 `key` 时，即使数据项的内容改变了，`key` 仍然保持不变，导致 React 无法正确识别和优化渲染。
2. **性能问题：**当列表项发生变动时，如果使用 `index` 作为 `key`，React 可能会做更多的 DOM 操作来更新视图，因为它无法准确地通过 `key` 识别哪些元素是新的，哪些元素被移动了位置。这可能导致不必要的重渲染和性能下降。
3. **组件状态的问题：**对于使用 `state` 的组件，如果列表项的顺序改变，使用 `index` 作为 `key` 可能会导致状态错乱。例如，当你删除一个列表项时，后面的项会移上来，它们的 `index` 改变了，如果它们有独立的状态，这时会由于 `index` 作为 `key` 使得状态与视图匹配错误。

因此，推荐的做法是使用唯一的、稳定的标识符（如数据库中的 `id` 或者具有唯一性的 `hash` 值等）作为 `key`，这样无论数据如何变化，每个元素的 `key` 都是稳定的，可以帮助 React 更准确、更高效地进行 DOM 的比对和更新。

24.[React] 如何避免使用 context 的时候， 引起整个挂载节点树的重新渲染

关键词：React Context 渲染问题

要避免在 React 开发中使用 `context` 时引起整个挂载节点树的重新渲染，可以采取以下方法：

1. **React Context 数据分割：**把提供 `context value` 的部分提取到单独的组件中，并且仅在该组件中修改 `context value`。这样，当 `context value` 变化时，只有真正使用该 `context` 的消费组件会重新渲染，而非所有挂载节点都会重新渲染。

假设我们有一个应用，需要管理主题颜色和用户信息两个不同的数据。

首先，创建两个 Context：

```
1 import React from "react";
2
3 // 创建主题颜色 Context
4 const ThemeContext = React.createContext({ theme: "light" });
5
6 // 创建用户信息 Context
7 const UserContext = React.createContext({ user: null });
```

在顶层组件中，提供这两个 Context 的 Provider，并设置相应的值：


```
1 class App extends React.Component {
2   state = {
3     theme: "dark",
4     user: { name: "John Doe", age: 25 },
5   };
6
7   render() {
8     return (
9       <ThemeContext.Provider value={this.state.theme}>
10        <UserContext.Provider value={this.state.user}>
11          <Toolbar />
12        </UserContext.Provider>
13      </ThemeContext.Provider>
14    );
15  }
16 }
```

然后，在需要使用主题颜色的组件中，可以通过以下方式获取：

```
1 class ThemedButton extends React.Component {
2   static contextType = ThemeContext;
3
4   render() {
5     const theme = this.context;
6     return <Button theme={theme} />;
7   }
8 }
```

在需要使用用户信息的组件中，同样方式获取：

```
1 class UserProfile extends React.Component {
2   static contextType = UserContext;
3
4   render() {
5     const user = this.context;
6     return (
7       <div>
8         <p>用户名：{user.name}</p>
9         <p>年龄：{user.age}</p>
10      </div>
11    );
12  }
```


在上述例子中，我们将主题颜色 and 用户信息分割到不同的 Context 中。`ThemeContext` 用于传递主题相关的数据，`UserContext` 用于传递用户相关的数据。这样，不同的组件可以根据自己的需求订阅相应的 Context，获取所需的数据，而不会相互干扰。每个组件只需要关注自己所使用的 Context，提高了代码的可读性和可维护性。同时，当某个 Context 的数据发生变化时，只有订阅了该 Context 的组件才会重新渲染，避免了不必要的重新渲染。

2. 对消费组件使用 `React.memo()` 进行包裹：`React.memo` 可以对函数组件进行浅比较，如果组件的 props 没有变化，就不会触发重新渲染。通过将消费 `context` 的组件用 `React.memo()` 包裹，可以避免不必要的重新渲染。

例如，假设有一个 `ContextProvider` 组件提供 `context value`，以及一个使用该 `context` 的子组件 `ConsumerComponent`，优化后的代码可能如下所示：

```
1  const ContextProvider = ({ children }) => {
2    // 管理 context value 的状态
3    const [value, setValue] = useState(/* 初始值 */);
4
5    return <MyContext.Provider value={value}>{children}</MyContext.Provider>;
6  };
7
8  const ConsumerComponent = React.memo(({ contextValue }) => {
9    // 仅根据 context value 进行渲染或处理逻辑
10   return <div>{/* 使用 context value 的相关逻辑 */}</div>;
11 });
```

在上述示例中，`ContextProvider` 负责管理 `context value` 的状态变化，而 `ConsumerComponent` 是使用 `context` 的消费组件，并通过 `React.memo()` 进行了包裹。这样，当 `value` 发生变化时，只有 `ConsumerComponent` 会根据浅比较来决定是否重新渲染，而不是整个挂载节点树都重新渲染。

通过以上方式，可以减少使用 `context` 时不必要的重新渲染，提高应用的性能。但具体的优化策略还需要根据项目的实际情况进行选择 and 调整。同时，还需注意避免在 `context` 中传递过于复杂 or 频繁变化的数据，以减少不必要的渲染次数。

25.[React] useState 的原理是什么，背后怎么执行的

关键词：useState 的原理

`useState` 是 React 库中的一个 Hook，它允许你在函数组件中添加 React 状态。使用 `useState`，你可以给组件添加内部状态，并且能够通过调用这个 Hook 来更新状态，从而触发组件

的重新渲染。

原理简述

`useState` 的工作原理基于 React 的渲染机制和 Fiber 架构。以下是 `useState` 工作流程的简要概述：

- 初始化状态：当你在函数组件中调用 `useState` 时，React 会为该组件创建一个状态变量。如果提供了初始值，状态将被初始化为该值。
- 返回更新函数：`useState` 返回一个数组，包含当前的状态值和一个更新该状态的函数（通常命名为 `setState`）。
- 调用更新函数：当你调用这个更新函数并传入一个新的状态值时，React 会将这个新的状态值与当前状态合并，并计划重新渲染组件。
- 重新渲染：在下一次的渲染周期中，React 会使用新的状态值重新渲染组件。
- 状态持久化：React 通过内部机制确保状态在组件的多次渲染之间保持不变。

执行过程

以下是 `useState` 在 React 内部可能的执行过程：

- 调用 `useState`：在函数组件中调用 `useState(initialState)`。
- 创建状态对象：React 创建一个状态对象，存储状态值和与之关联的更新函数。
- 渲染组件：使用当前的状态值渲染组件。
- 更新状态：当组件需要更新状态时，调用由 `useState` 返回的更新函数，例如 `setState(newState)`。
- 调度更新：React 将更新调度到下一个渲染周期，并标记组件为需要重新渲染。
- 批量处理：React 可能会将多个状态更新批处理在一起，以避免不必要的多次渲染。
- 重新渲染组件：在下一个渲染周期，React 使用新的状态值重新渲染组件。
- 状态持久化：React 的状态持久化机制确保即使在组件卸载和重新挂载后，状态也能被正确地恢复。

代码示例

```
1 import React, { useState } from "react";
2
3 function Counter() {
4   const [count, setCount] = useState(0); // 初始化状态为 0
5   return (
6     <div>
7       <p>Count: {count}</p>
8       <button onClick={() => setCount(count + 1)}>Increment</button>
9     </div>
10  )
11 }
```


代码小迷妹

代码小迷妹

代码小迷妹

```
9    );
10 }
```

在这个例子中，`useState` 被用来初始化 `count` 状态，并提供了一个 `setCount` 函数来更新它。每次点击按钮时，`setCount` 被调用，React 计划重新渲染组件，并在下一次渲染周期中使用新的状态值。

26.如何避免使用 context 的时候， 引起整个挂载节点树的重新渲染【热度：420】

关键词：React Context 渲染问题

要避免在 React 开发中使用 `context` 时引起整个挂载节点树的重新渲染，可以采取以下方法：

1. React Context 数据分割：把提供 `context value` 的部分提取到单独的组件中，并且仅在该组件中修改 `context value`。这样，当 `context value` 变化时，只有真正使用该 `context` 的消费组件会重新渲染，而非所有挂载节点都会重新渲染。

假设我们有一个应用，需要管理主题颜色 and 用户信息两个不同的数据。

首先，创建两个 Context：

代码小迷妹

代码小迷妹

代码小迷妹

```
1 import React from "eact";
2
3 // 创建主题颜色 Contextconst ThemeContext = React.createContext({ theme: "light"
  });
4
5 // 创建用户信息 Contextconst UserContext = React.createContext({ user: null });
```

在顶层组件中，提供这两个 Context 的 Provider，并设置相应的值：

代码小迷妹

代码小迷妹

代码小迷妹

```
1 class App extends React.Component {
2   state = {
3     theme: "dark",
4     user: { name: "John Doe", age: 25 },
5   };
6   render() {return (
7     <ThemeContext.Provider value={this.state.theme}>
8       <UserContext.Provider value={this.state.user}>
9         <Toolbar />
10      </UserContext.Provider      </ThemeContext.Provider      >);
11   }
12 }
```

代码小迷妹

代码小迷妹

代码小迷妹

然后，在需要使用主题颜色的组件中，可以通过以下方式获取：

```
1 class ThemedButton extends React.Component {
2   static contextType = ThemeContext;
3   render() {
4     const theme = this.context;return <Button theme={theme} />;
5   }
6 }
```

在需要使用用户信息的组件中，同样方式获取：

```
1 class UserProfile extends React.Component {
2   static contextType = UserContext;
3   render() {
4     const user = this.context;return (
5       <div>
6         <p>用户名：{user.name}</p>
7         <p>年龄：{user.age}</p>
8       </div>
9     );
10  }
11 }
```

在上述例子中，我们将主题颜色 and 用户信息分割到不同的 Context 中。 ThemeContext 用于传递主题相关的数据， UserContext 用于传递用户相关的数据。这样，不同的组件可以根据自己的需求订阅相应的 Context，获取所需的数据，而不会相互干扰。每个组件只需要关注自己所使用的 Context，提高了代码的可读性和可维护性。同时，当某个 Context 的数据发生变化时，只有订阅了该 Context 的组件才会重新渲染，避免了不必要的重新渲染。

2. 对消费组件使用 React.memo() 进行包裹： React.memo 可以对函数组件进行浅比较，如果组件的 props 没有变化，就不会触发重新渲染。通过将消费 context 的组件用 React.memo() 包裹，可以避免不必要的重新渲染。

例如，假设有一个 ContextProvider 组件提供 context value，以及一个使用该 context 的子组件 ConsumerComponent，优化后的代码可能如下所示：

```
1 const ContextProvider = ({ children }) => {// 管理 context value 的状态const
  [value, setValue] = useState(/* 初始值 */);
2 return <MyContext.Provider value={value}>{children}</MyContext.Provider>;
3 };
4
```



```
5  const ConsumerComponent = React.memo(({ contextValue }) => {// 仅根据 context value 进行渲染或处理逻辑
  return <div>{/* 使用 context value 的相关逻辑 */}</div>;
6  });
```

在上述示例中，`ContextProvider` 负责管理 `context value` 的状态变化，而 `ConsumerComponent` 是使用 `context` 的消费组件，并通过 `React.memo()` 进行了包裹。这样，当 `value` 发生变化时，只有 `ConsumerComponent` 会根据浅比较来决定是否重新渲染，而不是整个挂载节点树都重新渲染。

通过以上方式，可以减少使用 `context` 时不必要的重新渲染，提高应用的性能。但具体的优化策略还需要根据项目的实际情况进行选择 and 调整。同时，还需注意避免在 `context` 中传递过于复杂或频繁变化的数据，以减少不必要的渲染次数。

27.UseEffect 依赖为空数组与 componentDidMount 区别

关键词：`useEffect` 与 `componentDidMount` 区别

`useEffect` 是 React 函数组件的生命周期钩子，它是替代类组件中 `componentDidMount`、`componentDidUpdate` 和 `componentWillUnmount` 生命周期方法的统一方式。

当你给 `useEffect` 的依赖项数组传入一个空数组（`[]`），它的行为类似于 `componentDidMount`，但实质上有些区别：

1. 执行时机：
 - `componentDidMount`：在类组件的实例被创建并插入 DOM 之后（即挂载完成后）会立即被调用一次。
 - `useEffect`（依赖为空数组）：在函数组件的渲染结果被提交到 DOM 之后，在浏览器绘制之前被调用。React 保证了不会在 DOM 更新后阻塞页面绘制。
2. 清除操作：
 - `componentDidMount`：不涉及清理机制。
 - `useEffect`：可以返回一个清理函数，React 会在组件卸载或重新渲染（当依赖项改变时）之前调用这个函数。对于只依赖空数组的 `useEffect`，此清理函数只会在组件卸载时被调用。
3. 执行次数：
 - `componentDidMount`：在 render 执行之后，`componentDidMount` 会执行，如果在这个生命周期中再一次 `setState`，会导致再次 render，返回了新的值，浏览器只会渲染第二次 render 返回的值，这样可以避免闪屏。
 - `useEffect`：是在真实的 DOM 渲染之后才会去执行，在这个 hooks 中再一次 `setState`，这会造成两次 render，有可能会闪屏。

实际上 `useLayoutEffect` 会更接近 `componentDidMount` 的表现，它们都同步执行且会阻碍真实的 DOM 渲染的

28.UseEffect 钩子的工作原理是什么【滴滴】

关键词：React useEffect

`useEffect` 钩子的工作原理涉及到 React 的渲染流程和副作用的调度机制。以下是其工作原理的详细说明：

- 调度副作用：当你在组件内部调用 `useEffect` 时，你实际上是将一个副作用函数及其依赖项数组排队等待执行。这个函数并不会立即执行。
- 提交阶段（Commit Phase）：React 渲染组件并且执行了所有的纯函数组件或类组件的渲染方法后，会进入所谓的提交阶段。在这个阶段，React 将计算出的新视图（新的 DOM 节点）更新到屏幕上。一旦这个更新完成，React 就知道现在可以安全地执行副作用函数了，因为这不会影响到正在屏幕上显示的界面。
- 副作用执行：提交阶段完成后，React 会处理所有排队的副作用。如果组件是首次渲染，所有的副作用都会执行。如果组件是重新渲染，React 会首先对比副作用的依赖项数组：如果依赖项未变，副作用则不会执行；如果依赖项有变化，或者没有提供依赖项数组，副作用会再次执行。
- 清理机制：如果副作用函数返回了一个函数，那么这个函数将被视为清理函数。在执行当前的副作用之前，以及组件卸载前，React 会先调用上一次渲染中的清理函数。这样确保了不会有内存泄漏，同时能撤销上一次副作用导致的改变。
- 延迟副作用：尽管 `useEffect` 会在渲染之后执行，但它是异步执行的，不会阻塞浏览器更新屏幕。这意味着 React 会等待浏览器完成绘制之后，再执行你的副作用函数，以此来确保副作用处理不会导致用户可见的延迟。

通过这种机制，`useEffect` 允许开发者以一种优化的方式来处理组件中可能存在的副作用，而不需要关心渲染的具体时机。退出清理功能确保了即使组件被多次快速创建和销毁，应用程序也能保持稳定和性能。

29.介绍一下 useReducer【滴滴】

关键词：React useReducer

`useReducer` 是 React Hooks 的一个部分，它为状态管理提供了一个更加灵活的方法。`useReducer` 特别适合处理包含多个子值的复杂状态逻辑，或者当下一个状态依赖于之前的状态时。与 `useState` 相比，`useReducer` 更适合于复杂的状态逻辑，它使组件的状态管理更加清晰和可预测。

基础使用：

```
1 const [state, dispatch] = useReducer(reducer, initialState);
```


- `state`：当前管理的状态。
- `dispatch`：一个允许你分发动作(action)来更新状态的函数。
- `reducer`：一个函数，接受当前的状态和一个动作对象作为参数，并返回一个新的状态。
- `initialState`：初始状态值。

Reducer 函数：

Reducer 函数的格式如下：

```
1 function reducer(state, action) {
2   switch (action.type) {
3     case "ACTION_TYPE": {
4       // 处理动作并返回新的状态
5       return newState;
6     }
7     // 更多的动作处理
8     default:
9       return state;
10  }
```

动作（Action）：

动作通常是一个包含 `type` 字段的对象。`type` 用于在 reducer 函数中标识要执行的动作。动作对象也可以包含其他数据字段，用于传递动作所需的额外信息。

示例：

以下是一个使用 `useReducer` 的简单示例：

```
1 import React, { useReducer } from "react";
2
3 // 定义reducer函数
4 function counterReducer(state, action) {
5   switch (action.type) {
6     case "increment":
7       return { count: state.count + 1 };
8     case "decrement":
9       return { count: state.count - 1 };
10    default:
11      return state;
12  }
13 }
14
15 function Counter() {
16   // 初始化状态和dispatch函数
```



```
17   const [state, dispatch] = useReducer(counterReducer, { count: 0 });
18
19   return (
20     <>
21       Count: {state.count}
22       <button onClick={() => dispatch({ type: "decrement" })}>-</button>
        <button onClick={() => dispatch({ type: "increment" })}>+</button></>
23     );
24   }
```

代码小迷妹

代码小迷妹

代码小迷妹

在上面的例子中，我们创建了一个简单的计数器。当用户点击按钮时，会分发一个包含 `type` 的动作到 `useReducer` 钩子。然后，`reducer` 函数根据动作 `type` 来决定如何更新状态。

使用场景：

代码小迷妹

- 管理局部组件的状态。

代码小迷妹

代码小迷妹

- 处理复杂的状态逻辑。
- 当前状态依赖上一状态时，可以通过上一状态计算得到新状态。

`useReducer` 通常与 `Context` 一起使用可以实现不同组件间的状态共享，这在避免 prop drilling（长距离传递 prop）的同时使状态更新更为模块化。

30.[React] 是如何处理组件更新和渲染的？【快手】

代码小迷妹

代码小迷妹

代码小迷妹

React 组件的更新和渲染遵循一个相对严格的生命周期，这个生命周期在 React 16 版本之后，也就是从引入 Fiber 架构开始，稍微有所变化。React 通过一系列的生命周期方法以及新引入的 Hooks API，对组件的更新进行管理，主要流程如下：

类组件的生命周期方法包括：

1. 挂载(Mounting)

代码小迷妹

- `constructor()` : 组件被创建时调用，初始化 state。

代码小迷妹

- `static getDerivedStateFromProps()` : 组件实例化后和接受新属性时将会调用。
- `render()` : 唯一必须实现的方法，返回元素描述。
- `componentDidMount()` : 组件挂载（插入 DOM 树中）之后调用。

2. 更新(Updating)

代码小迷妹

- `static getDerivedStateFromProps()` : 在接收到新的 props 时调用。
- `shouldComponentUpdate()` : 在接收到新的 props 或者 state 时，决定是否进行渲染。
- `render()` : 重新渲染组件。
- `getSnapshotBeforeUpdate()` : 在最新的渲染输出提交到 DOM 前将会立即调用。

代码小迷妹

代码小迷妹

- `componentDidUpdate()` : 在组件更新后调用。

3. 卸载(Unmounting)

- `componentWillUnmount()` : 在组件卸载及销毁之前直接调用。

React 16.3 之后的生命周期的变化

React 团队增加了新的生命周期方法，并且准备弃用某些旧的生命周期方法（如 `componentWillMount`、`componentWillReceiveProps`、`componentWillUpdate` 等）。引入了如 `static getDerivedStateFromProps` 和 `getSnapshotBeforeUpdate` 等新的生命周期方法。

函数组件和 Hooks

在 React 16.8 版本后，引入了 Hooks API，允许在不编写类的情况下使用 state 以及其他的 React 特性。对于函数组件，有几个常用的 Hooks：

- `useState` : 在函数组件中添加 state。
- `useEffect` : 可以在组件中执行副作用操作（数据请求、订阅以及手动更改 React 组件中的 DOM 等）。
- `useContext` : 允许你访问 React 的 Context 对象。
- `useReducer` : 另一种在组件中管理 state 的方式，它用于复杂的 state 逻辑。
- 其他 Hooks（如 `useCallback`，`useMemo`，`useRef` 等）。

更新和渲染流程：

1. 当组件的 state 或者 props 发生变化时，React 会将新的 props 和 state 比较之前的，根据比较结果决定是否进行更新。
2. 如果 `shouldComponentUpdate`、`PureComponent` 或 `React.memo` 表示不需要更新，React 将不会进行更新。
3. 如果需要更新，React 会调用 `render` 方法以及相关的生命周期方法或 Hooks，这个过程会创建一个虚拟 DOM 树。
4. React 之后会对比新的虚拟 DOM 树与上一次更新时的虚拟 DOM 树，通过 DOM diffing 算法判断在哪进行实际的 DOM 更新。
5. 应用必要的 DOM 更新到实际的 DOM 树上，如果有必要，调用 `getSnapshotBeforeUpdate` 和 `componentDidUpdate` 方法。

这个过程保持了 React 组件的高效和可预测性，同时提供了生命周期的方法和 Hooks，使开发者能够插入自定义行为或响应组件的生命周期事件。

31.[React] forwardsRef 作用是啥， 有哪些使用场景？ 【PDD】

关键词：forwardsRef 作用、forwardsRef 使用场景

在 React 中，`forwardRef` 是一个用来传递 `ref` 引用给子组件的技术。通常情况下，`refs` 是不会透传给子组件的，因为 `refs` 并不是像 `props` 那样的属性。`forwardRef` 提供了一种机制，可以将 `ref` 自动地通过组件传递到它的子组件。

`forwardRef` 的作用：

- 访问子组件的 DOM 节点： 当需要直接访问子组件中的 DOM 元素（例如，需要管理焦点或测量尺寸）时，可以使用 `forwardRef`。
- 在高阶组件（HOC）中转发 refs: 封装组件时，通过 `forwardRef` 可以将 `ref` 属性透传给被封装的组件，这样父组件就能够通过 `ref` 访问到实际的子组件实例或 DOM 节点。
- 在函数组件中使用 refs(React 16.8+) ： 在引入 Hook 之前，函数组件不能直接与 `refs` 交互。但是，引入了 `forwardRef` 和 `useRef` 之后，函数组件可以接受 `ref` 并将它透传给子节点。

使用场景举例：

代码小迷妹

代码小迷妹

代码小迷妹

1. 访问子组件的 DOM 节点

假设你有一个 `FancyButton` 组件，你想从父组件中直接访问这个按钮的 DOM 节点。

代码小迷妹

代码小迷妹

代码小迷妹

```
1 const FancyButton = React.forwardRef((props, ref) => (  
2   <button ref={ref} className="FancyButton">  
3     {props.children}  
4   </button>  
5 ));  
6  
7 // 现在你可以从父组件中直接获取DOM引用  
8 const ref = React.createRef();  
9 <FancyButton ref={ref}>Click me!</FancyButton>
```

2. 在高阶组件中转发 refs

代码小迷妹

代码小迷妹

代码小迷妹

一个常见的模式是为了抽象或修改子组件行为的高阶组件（HOC）。`forwardRef` 可以用来确保 `ref` 可以传递给包装组件：

代码小迷妹

代码小迷妹

代码小迷妹

```
1 function logProps(Component) {  
2   class LogProps extends React.Component {  
3     componentDidMount(prevProps) {  
4       console.log("old props:", prevProps);  
5       console.log("new props:", this.props);  
6     }  
7  
8     render() {  
9       const { forwardedRef, ...rest } = this.props;  
10
```


代码小迷妹

代码小迷妹

代码小迷妹

```
11      // 将自定义的 prop 属性 "forwardedRef" 定义为 refreturn <Component ref=
    {forwardedRef} {...rest} />;
12    }
13  }
14
15  // 注意：React.forwardRef 回调的第二个参数 "ref" 传递给了LogProps组件的
    props.forwardedRef
代码小迷妹16    return React.forwardRef((props, ref) => {                                代码小迷妹
17      return <LogProps {...props} forwardedRef={ref} />;
18    });
19  }
```

3. 在函数组件中使用 ref

在 Hook 出现之前，函数组件不能够直接与 `ref` 交云。现在可以这样做：

代码小迷妹

代码小迷妹

代码小迷妹

```
1  const MyFunctionalComponent = React.forwardRef((props, ref) => {
2    return <input type="text" ref={ref} />;
3  });
4
5  const ref = React.createRef();
6  <MyFunctionalComponent ref={ref} />
```

代码小迷妹

代码小迷妹

代码小迷妹

当你在父组件中控制子组件中的 DOM 元素或组件实例的行为时，`forwardRef` 是非常有用的工具。不过，如果可行的话，通常最好通过状态提升或使用 context 来管理行为，只有在没有其他替代的情况下才选择使用 refs。